DIAMONDIQ

Codebase Overview

Repository inventory, architecture, testing, coverage, dependencies, and maintainability snapshot

Prepared July 31, 2026

617 PROJECT FILES +235 since Jul 18	**486** CLASSIFIED CODE FILES +46.4%	**58,514** CLASSIFIED CODE LINES +38.1%
2.10 MiB CLASSIFIED CODE +46.2%	**529** PASSING TESTS +126	**Clean** PRE-REPORT WORKTREE main @ 52f10c8f

DiamondIQ has grown from a local-first baseball analytics application into an authenticated baseball intelligence workspace. The Rails JSON API and PostgreSQL model still anchor MLB and Statcast ingestion, derived analytics, and benchmarks; the Vue application now adds decision-support workflows, owned records, saved analyses, collaborative notes, audit history, and role-aware administration.

WHAT CHANGED SINCE THE REFERENCE REPORT

- Added accounts, bearer-token sessions, five staff roles, administrator controls, ownership policies, and audit logs.
- Expanded decision support with Watchlists, acquisition need profiles, fit scoring, candidate alternatives, lineup scenarios, opponent reports, trend alerts, and player comparisons.
- Added saved analyses, notes and tags, schedules, standings, storage audit/backup tooling, and more durable background-task workflows.
- Reduced AdminView.vue from 3,212 to 1,734 lines by extracting focused components, composables, and formatting helpers.

> Quality signal
> All 392 Rails examples and 137 Vitest tests passed. Backend line coverage is 93.03%. Frontend coverage is 80.94% lines and 69.23% branches; most of the new gap is concentrated in recently added workflow code.

STACK AND DATA FLOW

Backend	Frontend	Primary flow
Ruby 3.2.3 / Rails 7.1.6 / PostgreSQL / Solid Queue / RSpec	Vue 3.5.35 / Vue Router 4.6 / Vite 8 / Vitest 4.1.8	MLB Stats API + Baseball Savant + local CSV -> sync/import services -> PostgreSQL -> analytics/benchmarks -> Rails API -> Vue

Architecture and capability surface

The application remains a two-application monorepo, but its responsibilities now span data ingestion, analytical read models, decision workflows, collaboration, authorization, and operations.

SYSTEM FLOW

Sources	Ingestion	Core data	Derived layer	Delivery
MLB Stats API Baseball Savant Local CSV	Downloaders Importers Sync boundaries Solid Queue jobs	PostgreSQL 56 tables 53 migrations Raw payload lineage	Daily analytics Benchmarks Percentiles Trend events	Rails JSON API Query objects Vue routes Role-aware UI

CAPABILITY MAP

Area	Current responsibilities	Primary implementation
Baseball data	Schedules, rosters, profiles, transactions, games, box scores, season stats, Statcast pitches	Downloaders/importers, sync services, jobs, canonical models
Analytics	Leaderboards, profiles, game analysis, rolling trends, contextual benchmarks, standings, playoff odds	19 query objects, 66 services, versioned daily summaries
Decision support	Opponent preparation, lineup scenarios/recommendations, acquisition needs, fit scoring, alternatives	Owned workflow controllers, policies, scoring services, dedicated views
Collaboration	Saved analyses, staff notes, organization tags, revision history, workflow audit trails	Users, notes/tags, saved records, audit logs, authorization concerns
Operations	Persisted Admin tasks, progress/cancel, data health, storage audit, backup/restore tests	Solid Queue jobs, task launchers, Admin components, Rake tasks

DELIVERY SURFACE

26	17	15	3
RAILS CONTROLLERS +12	VUE ROUTE RECORDS +10	VUE VIEWS +9	AUTHORIZATION POLICIES plus auth concern

Architectural center of gravity

The highest-risk boundaries are now authorization/ownership, durable background work, and cross-feature state restoration. Query objects continue to protect analytical read contracts, while newer workflow controllers and composables coordinate more mutable state.

Repository profile

The source-oriented inventory excludes dependencies, Git internals, datasets, builds, logs, storage, temporary files, editor metadata, coverage output, and the report output directory.

179	79	135
BACKEND APP FILES	FRONTEND APP FILES	TEST/SUPPORT FILES
+62	+40	+31
17,654	**30.2%**	**57.16 MiB**
TEST/SUPPORT LINES	LINES IN TESTS	PROJECT MATERIALS
+4,087	was 32.0%	54.02 MiB in docs

CLASSIFIED CODE BY MAJOR TYPE

Type	Files	Lines	Share of lines	Change vs Jul 18
Ruby	355	34,466	58.9%	+9,012
Vue SFC	46	11,832	20.2%	+3,371
JavaScript	67	10,217	17.5%	+3,427
CSS	2	1,051	1.8%	+100
Rake	12	660	1.1%	+151
Shell + SQL	4	288	0.5%	new/expanded

ARCHITECTURE COUNTS

Rails models	47	Rails controllers	26	Services	66
Queries	19	Serializers	3	Jobs	10
Policies	3	Vue views	15	Vue components	30
Composables	25	Database tables	56	Migrations	53
Rake task files	12	RSpec spec files	97	Vitest spec files	35

Inventory interpretation

Classified code grew 38.1% in thirteen days, while test/support lines grew 30.1%. The repository also added a large screenshot/reference corpus: docs now accounts for 54.02 MiB, including 43.82 MiB of screenshots. Code footprint remains 2.10 MiB.

Concentration and maintainability

Large files are not automatically defects, but they concentrate review surface, conditional behavior, and regression risk. This ranking uses bytes, with line counts as context.

#	File	Size	Lines	Area
1	frontend/src/views/TeamProfileView.vue	77.3 KiB	1,294	Frontend view
2	backend/db/schema.rb	61.6 KiB	1,274	Generated schema
3	frontend/src/views/GameSummaryView.vue	59.2 KiB	944	Frontend view
4	frontend/src/views/PlayerProfileView.vue	49.6 KiB	1,722	Frontend view
5	frontend/src/views/AdminView.vue	41.6 KiB	1,734	Frontend view
6	frontend/src/views/WatchlistsView.vue	39.2 KiB	658	Frontend view
7	backend/app/queries/team_profile_snapshot_query.rb	38.1 KiB	1,157	Backend query
8	frontend/src/views/__tests__/AdminView.spec.js	36.0 KiB	859	Frontend test
9	frontend/src/views/__tests__/GameSummaryView.spec.js	29.5 KiB	629	Frontend test
10	frontend/src/components/PlayerSeasonStatsDashboard.vue	27.5 KiB	841	Frontend component

CONCENTRATION SIGNAL

459.6 KiB
TEN LARGEST FILES
was 405.2 KiB

21.3%
SHARE OF CODE BYTES
was 27.4%

-46.0%
ADMINVIEW LINE CHANGE
3,212 -> 1,734

WHAT THE RANKING SAYS NOW

- The Admin decomposition succeeded: its byte rank moved from first to fifth, and the frontend gained reusable Admin components and workflow composables.
- TeamProfileView.vue is now the clearest presentation hotspot. It combines dashboard tabs, opponent preparation, lineup planning, leaders, roster state, and saved-view behavior.
- PlayerProfileView.vue has the highest hand-written line count among the top files; WatchlistsView.vue is smaller but carries dense mutable workflow state and weak coverage.
- The two profile query objects remain valuable contract boundaries, but team-profile aggregation is large enough to justify internal domain helpers and focused contract tests.

SUGGESTED DECOMPOSITION ORDER

Priority	Target	Practical seam
1	TeamProfileView.vue	Extract opponent-prep, lineup-planner, leaders, and tab-state modules with route/state contract tests.
2	PlayerProfileView.vue	Split biography, trend analysis, season table, external links, notes, and saved-analysis concerns.
3	WatchlistsView.vue	Move discovery, candidate review, transition forms, alternatives, and ownership/audit panels into focused units.
4	team_profile_snapshot_query.rb	Extract reusable stat aggregation and opponent-preparation composition without changing response shape.

Quality signal

Both suites pass. Coverage was measured during this refresh with Ruby's built-in Coverage API for backend application/lib lines and Vitest's V8 provider for frontend source files.

392 RAILS EXAMPLES PASSED +83	**137** FRONTEND TESTS PASSED +43	**529** TOTAL PASSING TESTS +31.3%
93.03% BACKEND LINE COVERAGE 8,050 / 8,653	**80.94%** FRONTEND LINE COVERAGE 2,830 / 3,496	**69.23%** FRONTEND BRANCH COVERAGE 3,678 / 5,312

COVERAGE SUMMARY

Suite	Metric	Covered / total	Coverage	Prior snapshot	Change
Rails backend	Lines	8,050 / 8,653	93.03%	93.48%	-0.45 pp
Vue frontend	Lines	2,830 / 3,496	80.94%	90.74%	-9.80 pp
Vue frontend	Statements	3,134 / 4,112	76.21%	85.56%	-9.35 pp
Vue frontend	Functions	959 / 1,275	75.21%	86.47%	-11.26 pp
Vue frontend	Branches	3,678 / 5,312	69.23%	73.19%	-3.96 pp

INTERPRETATION

- Backend line coverage stayed effectively flat while measured executable lines increased by roughly 51%, a strong signal that service/query growth was accompanied by tests.
- Frontend coverage fell because the measured surface expanded faster than exercised behavior. The largest gaps sit in new synchronization, ownership, collaboration, and Watchlists workflows rather than legacy analytical views.
- The frontend has 1,634 uncovered measured branches. Branch coverage remains the best aggregate indicator for untested loading, authorization, error, cancellation, empty-state, and transition paths.
- Coverage is a risk signal, not proof of correctness. Authorization isolation, audit attribution, background-task recovery, and partial import behavior deserve scenario-level verification even when line coverage is high.

TEST INVENTORY

Area	Spec files	Passing cases	Observed strength
Rails	97	392	Broad request, service, query, job, and model coverage; zero failures or pending examples.
Vue	35	137	Strong analytical-view coverage; newer stateful workflow composables and controls are uneven.
Combined support	135 files	17,654 lines	30.2% of classified code lines are test/support code.

Comparability note
The coverage denominator changed with the codebase. Percentage declines should be read alongside the much larger measured surface and the 126 additional passing tests.

Where additional tests pay off

The tables rank files with at least 20 executable or measured lines. They identify concentration of uncovered behavior, not defects.

LOWEST BACKEND LINE COVERAGE

File	Covered / total	Coverage
app/services/pitch_data_sync_task_launcher.rb	8 / 27	29.63%
app/controllers/api/need_profiles_controller.rb	18 / 41	43.90%
app/services/pitch_data_sync_progress_tracker.rb	22 / 44	50.00%
app/services/mlb_game_details_downloader.rb	24 / 38	63.16%
app/services/mlb_player_profiles_downloader.rb	29 / 43	67.44%
app/services/mlb_player_transactions_downloader.rb	34 / 49	69.39%
app/services/mlb_roster_downloader.rb	37 / 52	71.15%
app/services/mlb_schedule_downloader.rb	39 / 54	72.22%
app/services/admin_import_task_launcher.rb	21 / 27	77.78%
app/controllers/api/watchlist_entries_controller.rb	54 / 67	80.60%

LOWEST FRONTEND LINE COVERAGE

File	Lines	Branches	Functions
src/composables/usePitchDataSync.js	4.94%	2.25%	0.00%
src/components/SavedAnalysisControls.vue	38.46%	21.88%	0.00%
src/composables/useAuth.js	42.50%	36.36%	44.44%
src/views/WatchlistsView.vue	49.55%	67.34%	27.08%
src/composables/usePlayerSeasonStatsImport.js	54.17%	36.00%	83.33%
src/composables/usePitchDataImport.js	57.14%	33.33%	83.33%
src/composables/useNotes.js	57.35%	54.55%	56.52%
src/composables/useBackgroundTaskRun.js	58.46%	54.46%	76.47%
src/components/NotesPanel.vue	59.21%	63.29%	48.48%
src/composables/useSavedAnalyses.js	64.00%	65.62%	57.14%

HIGHEST-VALUE SCENARIOS

Priority	Scenario cluster	Why it matters
1	Pitch sync lifecycle	Launcher validation, progress recovery, cancellation, retry, stale task state, malformed results, and partial imports.
2	Authorization and ownership	Cross-user reads/writes, administrator override, signed-out behavior, role changes, direct URL access, and audit attribution.
3	Saved analyses and notes	Visibility modes, URL/state restoration, delete/archive flows, revision history, tags, and failed requests.
4	Watchlist acquisition workflow	Discovery filters, fit recalculation, review transitions, assignment, alternatives, empty states, and concurrent refresh.
5	External data boundaries	Timeouts, non-200 responses, invalid JSON/CSV, rate limits, duplicate payloads, and idempotent replay.

Operational profile and next actions

Installed dependencies and repository history dominate disk usage. The codebase is healthy under test, but its fastest-growing risks are workflow branching, frontend concentration, and artifact/history growth.

DISK FOOTPRINT

Area	Disk size	Notes
Project materials (source-oriented)	57.16 MiB	Includes documentation and reference assets; excludes datasets, dependencies, Git, temp, coverage, and output.
Classified code	2.10 MiB	486 code files and 58,514 lines.
Documentation/reference assets	54.02 MiB	Includes 43.82 MiB of screenshots and an 8.7 MiB layered design source.
Frontend node_modules	95.16 MiB	Installed development and runtime packages.
Backend vendor	94.21 MiB	Vendored Ruby bundle footprint.
Git metadata	601.40 MiB	Repository objects/history; materially larger than the working source tree.

DEPENDENCY PROFILE

Stack	Direct runtime	Direct dev/test	Locked packages	Core packages
Rails backend	7	3	99 specs / 97 names	Rails, PostgreSQL, Puma, Solid Queue, RSpec, SeedFu
Vue frontend	2	6	269 package entries	Vue, Vue Router, Vite, Vitest, Vue Test Utils, jsdom

RECOMMENDED NEXT ACTIONS

#	Action	Outcome
1	Close workflow coverage gaps	Add focused tests for pitch sync, auth/ownership, saved analyses, notes, and Watchlists. Start with the files on page 6.
2	Persist coverage in CI	Declare @vitest/coverage-v8, add a maintained Rails coverage setup, publish artifacts, and use gradual per-area thresholds.
3	Continue frontend decomposition	Prioritize TeamProfileView, PlayerProfileView, and WatchlistsView while preserving route/state and API-contract tests.
4	Strengthen authorization contracts	Create a shared matrix of roles, ownership, visibility, and administrator override; exercise it through request and route-guard tests.
5	Control repository growth	Review large historical objects and keep generated datasets, screenshots, coverage, backups, and deploy artifacts outside Git where practical.
6	Track structural trends	Record code lines, largest files, coverage, test counts, migrations, and dependency/history footprint on a regular CI cadence.

METHODOLOGY AND LIMITATIONS

Inventory was measured from the working tree on branch main at commit 52f10c8f, which was clean before regenerating this report. Code classification includes Ruby, Rake, Vue, JavaScript, CSS, SQL, shell, and executable project scripts. It excludes .git, node_modules, vendor, data, dist, output, logs, storage, tmp, coverage, editor metadata, and local runtime caches. Backend coverage used Ruby's built-in line coverage while running the full RSpec suite against the local PostgreSQL test database. Frontend coverage used Vitest's V8 provider. Counts are a point-in-time snapshot and may change with generated schema, dependency installation, test environment, or classification rules.